📘 DSIP Module 5

Morphology & Image Compression

Complete Revision Guide

Every Concept · Every Numerical · Every Mnemonic

📚 **Subject:** Digital Signal & Image Processing (DSIP)

📖 **Module:** 5 — Introduction to Morphology and Image Compression

🎯 **Coverage:** All syllabus topics with worked numericals

⏱️ **Best used:** 1 day before semester end exam

⭐ **Star ratings:** ⭐⭐⭐ Guaranteed ⭐⭐ Likely ⭐ Possible

📑 Table of Contents

Ultrasound Image Enhancement

Final: Master Summary & Exam Strategy

⚡ Session 1: Dilation & Erosion

The foundation of all morphological operations

🧱 Prerequisites

Binary Image

An image where each pixel is either:

- **1 = BLACK** (the object/shape)
- **0 = WHITE** (background)

```
0 0 0 0 0          . . . . .
0 1 1 1 0    =     . ■ ■ ■ .       (■ = black, . = white)
0 1 1 1 0          . ■ ■ ■ .
0 0 0 0 0          . . . . .
```

Structuring Element (SE)

A tiny shape (like a 3×3 stamp) we slide across the image to modify it. The **origin** is the reference/anchor point.

```
SE example (plus shape):
. ■ .
■ ⊙ ■        ⊙ = the ORIGIN (anchor point)
. ■ .
```

🌅 DILATION (⊕) — Making things FATTER

> 🎯 **Essence:** *Dilation = GROW the object. Slide the SE over the image — wherever there's **ANY overlap**, turn the center pixel ON (black).*

> 🧠 **Memory Trick: "Dilation = Donation"** — each black pixel donates the SE shape around itself. The ⊕ symbol looks like a "plus/grow" sign.

Formula

$$A \oplus B = \{ x \mid (\hat{B})_x \cap A \neq \emptyset \}$$

"Set of all positions where the reflected SE, when shifted to x, has at least one overlap with A."

The "Stamp" View (Easier)

Place the SE on each black pixel of the image. Whatever cells the SE covers → those become black in the output. Combine all stamps.

Worked Mini-Example

Image: ONE single black pixel

```
. . . . .
. . . . .
. . ■ . .       ← single black dot at (2,2)
. . . . .
. . . . .
```

SE (plus shape):

```
. ■ .
■ ⊙ ■
. ■ .
```

After dilation:

```
. . . . .
. . ■ . .   ← SE's top arm
. ■ ■ ■ .   ← SE's middle row
. . ■ . .   ← SE's bottom arm
. . . . .
```

The single dot turned into a plus! ✅

What Dilation Is For (PDF slide 6)

- ✓ Repairs **BREAKS** in lines (gaps close up)
- ✓ Fills small **INTRUSIONS** (holes get filled)
- ⚠ Side effect: object becomes LARGER

🌑 EROSION (⊖) — Making things THINNER

> 🎯 **Essence:** *Erosion = SHRINK the object. Slide SE over image — only where the SE **completely fits inside** the object, keep that center pixel ON.*

> 🧠 **Memory Trick: "Erosion = Eraser"** — eraser only erases where it fits perfectly inside the shape. The ⊖ symbol looks like "minus/shrink".

The Algorithm

1. Place SE origin on each image pixel
2. Check: Do **ALL** ON-pixels of SE overlap with ON-pixels of image?
3. If YES (perfect fit) → output pixel = 1
4. If NO → output pixel = 0

Worked Mini-Example

Image: A 5×5 black square. **SE:** Plus shape.

```
Original 5×5 square:        After Erosion:
. . . . . . .               . . . . . . .
. ■ ■ ■ ■ ■ .               . . . . . . .   ← edge stripped
. ■ ■ ■ ■ ■ .               . . ■ ■ ■ . .
. ■ ■ ■ ■ ■ .        →      . . ■ ■ ■ . .   ← only inner 3×3 survives
. ■ ■ ■ ■ ■ .               . . ■ ■ ■ . .
. ■ ■ ■ ■ ■ .               . . . . . . .   ← edge stripped
. . . . . . .               . . . . . . .
```

What Erosion Is For

- ✓ Removes small noise specks
- ✓ Separates touching objects
- ✓ Shrinks objects
- ⚠ Side effect: can erase thin features completely

⚔️ Dilation vs Erosion

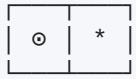
Property	Dilation ⊕	Erosion ⊖
Effect	GROWS object	SHRINKS object
Overlap rule	ANY overlap → ON	ALL must match → ON
Symbol	⊕ (plus)	⊖ (minus)
Repairs	Breaks, intrusions	Removes noise
Mnemonic	Donation 🎁	Eraser ✏️
Risk	Merges separate objects	Erases thin parts

🔑 GOLDEN RULE
DILATION: ANY pixel matches → keep ON
EROSION: ALL must match → keep ON

📖 Solved Numerical: Erosion (PDF Example 10.2)

Problem: Erode the letter "H"

SE used: 1×2 horizontal element with origin on left (both pixels ON)



```
⊙ = origin (ON), * = ON
So SE has 2 ON pixels horizontally
```

Logic: The output pixel survives only if the current pixel AND the pixel to its right are both ON in the original.

Step 1: Compare SE with input image. If all SE pixels overlap with image, origin pixel is turned ON.

Step 2: The structuring element is mapped along each row, shifted right by one pixel.

Step 3: Process repeats until last pixel.

Effect on H: The right vertical bar of H gets erased (its rightmost column has no pixel-to-the-right that's ON). H loses one column on its right side.

Refer PDF slide 16 for the final eroded H shape.

🎯 Likely Exam Questions

1. Define dilation and erosion. Write their mathematical equations. ⭐⭐⭐
2. Perform dilation/erosion on given image with given SE (numerical). ⭐⭐⭐
3. What are the applications of dilation and erosion? ⭐⭐
4. Differentiate dilation and erosion. ⭐⭐⭐

⚡ Session 2: Opening, Closing, Hit-or-Miss & Boundary Extraction

Compound morphological operations

🚪 OPENING (○) — Erode then Dilate

🎯 **Essence:** *Opening = Erosion FIRST, then Dilation. Smooths the object's outside and removes small noise specks.*

$$A \circ B = (A \ominus B) \oplus B$$

🧠 **Memory Trick: "OPENING opens by Eroding first"** — the FIRST operation tells you what it does. It cleans the OUTSIDE of objects.

Why It Works

- **Step 1 (Erode):** Shrinks object → small noise specks DISAPPEAR
- **Step 2 (Dilate):** Grows object back → main object recovers its size, but noise stays gone forever ✅

Effects

- ✓ Removes small noise specks
- ✓ Smooths outer boundary
- ✓ Breaks thin connections between objects
- ✓ Shape size roughly preserved

🚪 CLOSING (•) — Dilate then Erode

🎯 **Essence:** *Closing = Dilation FIRST, then Erosion. Fills small holes and gaps INSIDE the object.*

$$A \bullet B = (A \oplus B) \ominus B$$

🧠 **Memory Trick: "CLOSING closes the holes"** — fills gaps inside the shape. Order tip: **"C-D-E"** → **C**losing = **D**ilate then **E**rode.

Why It Works

- **Step 1 (Dilate):** Grows object → small holes/gaps GET FILLED
- **Step 2 (Erode):** Shrinks back → main object returns to original size, but filled holes STAY filled ✅

Effects

- ✓ Fills small holes inside object
- ✓ Connects nearby objects
- ✓ Smooths inner boundary
- ✓ Shape size roughly preserved

⚔️ Opening vs Closing

Property	Opening ○	Closing •
Order	Erode then Dilate	Dilate then Erode
Symbol	○ (open circle)	• (filled dot)
Removes	Outside noise	Inside holes
Affects	Bright details	Dark details
Mnemonic	Open up gaps	Close the holes

🔑 KEY PATTERN
OPENING: E → D (Erode first) → cleans OUTSIDE noise
CLOSING: D → E (Dilate first) → fills INSIDE holes

⚠️ **Common Mistake:** Mixing up the order! Memorize: Opening = "the eraser comes first" (erode); Closing = "the inflator comes first" (dilate).

🎯 HIT-OR-MISS TRANSFORM

🎯 **Essence:** *Hit-or-Miss = Find EXACT pattern matches in the image. Used for template matching.*

🧠 **Memory Trick: "HIT the foreground, MISS the background"** — both must match for a "hit".

The Big Idea

Find pixels that match a pattern AND have specific surrounding emptiness. We use TWO structuring elements:

- **B** = pattern of foreground (■) we want to HIT

- **W − B** = pattern of background (.) we want to MISS

Formula

$$HM(X, B) = (X \ominus B) \cap (X^c \ominus (W - B))$$

The 3-Step Process

1. **Erode the IMAGE with B** → finds where foreground fits
2. **Erode the COMPLEMENT (X^c) with (W − B)** → finds where background fits
3. **INTERSECT (AND)** the two results → only locations matching BOTH

Why Two Erosions?

A "hit" means:

- ✓ The foreground pixels look right (B fits inside the object)
- ✓ The background pixels look right (W−B fits inside the empty area)

If both happen at the same location → that's an EXACT match.

Applications

- Template matching (find specific shapes)
- Corner detection, endpoint detection
- Locating specific features (like the letter "T")

> ⚠️ **Common Mistake:** Forgetting to use the COMPLEMENT of the image for the second erosion. Tip: B looks at black, (W−B) looks at white.

📖 Solved Numerical: Hit-or-Miss (PDF Example 10.4)

Steps from PDF (slides 26-28)

Given: Input image (checkerboard-like pattern), structuring element B (foreground template), and W−B (background template).

Step 1: Erode the input image with B.

- Place B at each pixel; check if all ON pixels of B match ON pixels of image
- Result: Set of candidate ■ pixels (foreground matches)

Step 2: Erode the complement of input with (W−B).

- Take complement (flip 0s and 1s)
- Erode with (W−B)
- Result: Set of candidate ■ pixels (background matches)

Step 3: Intersect (AND) results from Steps 1 and 2.

- Only positions with BOTH conditions satisfied survive
- These are the EXACT pattern match locations

Refer PDF slide 28 for the final hit-or-miss output (small set of pixels showing exact matches).

🔲 BOUNDARY EXTRACTION

🎯 **Essence:** *Boundary = Original − Eroded version. Subtracting the eroded image from the original gives only the OUTLINE.*

$$\beta(A) = A - (A \ominus B)$$

🧠 **Memory Trick: "Strip the inside off — only the skin remains."** Erosion peels one layer; subtract from original = exactly that one peeled layer = boundary!

Visual

```
Original (filled):        After erosion:          Boundary (Original − Eroded):
■ ■ ■ ■ ■                 . . . . .               ■ ■ ■ ■ ■
■ ■ ■ ■ ■                 . ■ ■ ■ .               ■ . . . ■
■ ■ ■ ■ ■       −         . ■ ■ ■ .       =       ■ . . . ■
■ ■ ■ ■ ■                 . ■ ■ ■ .               ■ . . . ■
■ ■ ■ ■ ■                 . . . . .               ■ ■ ■ ■ ■

Filled square             Inside only             Outer shell only ✨
```

📝 Master Recap Table

Operation	Formula	Purpose
Dilation ⊕	$A \oplus B$	Grow object, fill gaps
Erosion ⊖	$A \ominus B$	Shrink object, remove noise
Opening ○	$(A \ominus B) \oplus B$	Remove outside noise
Closing •	$(A \oplus B) \ominus B$	Fill inside holes
Hit-or-Miss	$(X \ominus B) \cap (X^c \ominus (W-B))$	Exact template matching
Boundary	$A - (A \ominus B)$	Get object outline

🎯 Likely Exam Questions

1. Define Opening and Closing. Give equations and applications. ⭐⭐⭐
2. Differentiate Opening and Closing. ⭐⭐⭐
3. Explain Hit-or-Miss transform with example. ⭐⭐⭐
4. Apply Opening/Closing/Hit-or-Miss on given image (numerical). ⭐⭐⭐
5. Explain Boundary Extraction. ⭐⭐

⚡ Session 3: Compression Basics & Redundancies

Foundation of image compression theory

📦 What is Image Compression?

🎯 **Essence:** *Compression = Removing redundant data to make the file smaller, while keeping the image still useful.*

🧠 **Memory Trick: "Compression = Squeezing out the AIR (redundancy) from the image"** — like vacuum-packing clothes.

Two Flavors of Compression

Type	What Happens	Examples
LOSSLESS	Compressed → decompressed gives EXACT original. No quality loss.	Huffman, RLE, Arithmetic, LZW
LOSSY	Some info LOST forever. Slight quality drop, but much smaller files.	JPEG, MPEG, MP3, IGS, VQ

📐 Compression Ratio (C_R)

C_R = Uncompressed Size / Compressed Size = U_{size} / C_{size}

where U_{size} = M × N × k / 8 bytes
(M = rows, N = cols, k = bits per pixel; ÷8 to convert bits to bytes)

🧠 **Memory Trick:** CR is a ratio — Original ÷ Compressed. Higher CR = better compression. CR of 10 means 10× smaller (often written as 10:1).

📖 Solved Numerical: Compression Ratio (PDF slide 31)

Problem: 8-bit image, 256×256 pixels. After compression, file is 6,554 bytes. Find CR.

Step 1: Calculate uncompressed size

```
U_size = (256 × 256 × 8) / 8 = 65,536 bytes
         └─total bits──┘   └→ convert to bytes
```

Step 2: Apply formula

```
CR = 65,536 / 6,554 = 9.999 ≈ 10  →  10:1
```

Meaning: Original has 10 bytes for every 1 byte in compressed file. ✅

🔄 Relative Data Redundancy (R_D)

$$R_D = 1 - (1 / C_R)$$

Case	C_R	R_D	Meaning
$n_1 = n_2$ (same)	1	0	NO redundancy, no compression
$n_2 \ll n_1$ (compressed tiny)	∞	→ 1	MAX compression
$n_2 \gg n_1$ (compressed bigger)	0	→ −∞	Compression failed

n_1 = bits/symbols in ORIGINAL image; n_2 = bits/symbols in COMPRESSED image

> 💡 **Tip:** Smaller compressed file → bigger CR → R_D closer to 1 → good compression!
> Same size → CR = 1 → R_D = 0 → pointless
> Bigger compressed file → CR < 1 → R_D negative → disaster

🔍 Three Types of Redundancy ⭐⭐⭐

> 🧠 **Memory Trick:** Master Mnemonic: **"CIP"** = **C**oding, **I**nterpixel, **P**sychovisual

Redundancy	What It Is	Fix
1. CODING	Using more bits than needed per pixel. Ex: 8-bit image with only 16 gray levels → 4 bits enough!	Variable-length coding (Huffman, Arithmetic)
2. INTERPIXEL	Adjacent pixels are similar/correlated. Ex: blue sky → many similar pixels in a row	Run-Length Encoding, DPCM, Predictive coding
3. PSYCHOVISUAL	Eye doesn't notice some details (LOSSY). Ex: subtle color shifts humans can't perceive	Quantization (IGS), JPEG quantization

Real Examples

Coding redundancy example (PDF slide 35-36)

- 8 gray levels with non-uniform probabilities
- Code 1 uses 3 bits each → wasteful (L_{avg} = 3 bits)
- Code 2 uses variable lengths → frequent values get short codes (L_{avg} = 2.7 bits)
- CR = 3/2.7 = 1.11, R_D = 0.099 (about 10% redundancy removed)

Interpixel example (PDF slide 39)

- 1024 bits of binary data → can be encoded as run-length pairs in only 88 bits!
- Example: (1,63)(0,87)(1,37)(0,5)(1,4)(0,556)(1,62)(0,210) = 88 bits

Psychovisual example (PDF slide 42)

- 256 gray levels → reduced to 16 levels (8 bits → 4 bits) = **2:1 compression**
- Eye doesn't notice much, but image is smaller

📏 Fidelity Criteria — Measuring Quality Loss

Type	Description
A) Objective	Math-based. Measures pixel error using RMS error, SNR. Repeatable, exact.
B) Subjective	Human-based. Show image to viewers, get ratings. Average their opinions. Real-world quality but subjective.

Objective Fidelity Formulas (PDF slide 45)

Pixel error:

$$e(x,y) = \hat{f}(x,y) - f(x,y)$$

where $\hat{f}$ = compressed/decompressed, f = original

RMS Error:

$$e_{rms} = [\ (1/MN)\ \Sigma\ \Sigma\ (\hat{f}(x,y) - f(x,y))^2\ ]^{½}$$

Mean Square SNR:

$$SNR_{ms} = \Sigma\ \Sigma\ [\hat{f}(x,y)]^2 / \Sigma\ \Sigma\ [\hat{f}(x,y) - f(x,y)]^2$$

RMS SNR:

$$SNR_{rms} = \sqrt{(SNR_{ms})}$$

> 🧠 **Memory Trick:** "OBJECTIVE = math counts pixels"
> "SUBJECTIVE = humans count feelings"

🎯 Likely Exam Questions

1. What is image compression? Need for compression. ⭐⭐
2. Explain types of redundancies (Coding, Interpixel, Psychovisual). ⭐⭐⭐
3. Define compression ratio with example (numerical). ⭐⭐⭐
4. Differentiate lossless vs lossy compression. ⭐⭐⭐
5. Explain objective vs subjective fidelity criteria. ⭐⭐
6. Numerical: find CR and R_D given image size. ⭐⭐⭐

⚡ Session 4: Lossless Compression

Huffman · RLE · Arithmetic · DPCM — THE BIG NUMERICAL SESSION

🌳 Part A: HUFFMAN CODING ⭐⭐⭐

> 🎯 **Essence:** *Huffman = Give SHORTER codes to FREQUENT symbols, LONGER codes to RARE symbols. Reduces total bits.*

> 🧠 **Memory Trick: "Frequent = Short, Rare = Long"** — like how we say "the" (3 letters) million times but "rhinoceros" (10 letters) almost never.

Huffman Algorithm — 5 Steps

1. **SORT** symbols by probability (highest to lowest)
2. **COMBINE** the two LOWEST probabilities → one node (sum them)
3. **RE-SORT** the list with the new combined node
4. **REPEAT** until only ONE node remains (= 1.0)
5. **ASSIGN codes** by going back through tree:
    - Pick a convention (e.g., upper = 0, lower = 1)
    - Stay consistent throughout

DSIP Module 5 Revision Guide — Page 17 of 43

📖 Solved Numerical: Huffman Coding (PDF slides 37-38)

Problem: Image 10×10, 5-bit. Symbols and frequencies:

Symbol	Frequency	Probability
a2	40	0.40
a6	30	0.30
a1	10	0.10
a4	10	0.10
a3	6	0.06
a5	4	0.04
Total		**1.00**

Step 1-4: Source Reduction Table

```
SYMBOL  PROB    R1      R2      R3      R4
──────────────────────────────────────────
 a2    0.4     0.4     0.4     0.4    ┌─0.6
 a6    0.3     0.3     0.3    ┌─0.3   └ 0.4
 a1    0.1     0.1    ┌─0.2   └ 0.3
 a4    0.1    ┌─0.1   └ 0.1
 a3    0.06  ─┘ 0.1
 a5    0.04  ─┘

Merges performed:
R1: a3 (0.06) + a5 (0.04) = 0.1
R2: a4 (0.1)  + 0.1       = 0.2
R3: a1 (0.1)  + 0.2       = 0.3
R4: a6 (0.3)  + 0.3       = 0.6
Final: a2 (0.4) + 0.6     = 1.0 ✅
```

Step 5: Final Code Table

Symbol	Prob	Code	Length
a2	0.40	1	1
a6	0.30	00	2
a1	0.10	011	3
a4	0.10	0100	4
a3	0.06	01010	5
a5	0.04	01011	5

✓ a2 (most frequent) got shortest code (1 bit). a3 and a5 (rarest) got longest (5 bits).

The 4 Required Calculations

1. Average Code Length (L_{avg}):

```
L_avg = Σ (P_i × L_i)
      = 0.4(1) + 0.3(2) + 0.1(3) + 0.1(4) + 0.06(5) + 0.04(5)
      = 0.4 + 0.6 + 0.3 + 0.4 + 0.3 + 0.2
      = 2.2 bits/symbol
```

2. Total Bits to Transmit:

```
= (image size) × L_avg
= 10 × 10 × 2.2 = 220 bits
```

3. Entropy (theoretical minimum):

```
H = − Σ P_i × log₂(P_i)
H ≈ 2.1396 bits/symbol
(Huffman's L_avg is always ≥ entropy)
```

4. Percentage Saved:

```
= (Original bits − New bits) / Original bits × 100
= (10×10×5 − 10×10×2.2) / (10×10×5) × 100
= (500 − 220) / 500 × 100
= 56% saved ✅
```

Encoded String example: "a3 a1 a2 a2 a6" → 01010 011 1 1 00 = "010100111100"

⚠️ **Common Mistake:** 1. Forgetting to RE-SORT after combining
2. Inconsistent 0/1 assignment — pick one rule and stick with it
3. Not multiplying by image size for total bits

💡 **Tip:** Codes can vary based on convention (e.g., "1" or "0" for top branch) — both are valid as long as code **lengths** are correct and codes are **prefix-free**.

📊 Part B: RUN-LENGTH ENCODING (RLE)

🎯 **Essence:** *RLE = Replace runs of repeating values with (value, count) pairs.*

🧠 **Memory Trick: "AAAAA → A5"** — that's it. Count the runs.

Example (PDF slide 40)

```
Original:    a a a b c c c c       (8 symbols)
RLE encoded: a 3 b 1 c 4            (6 symbols)
```

Where it's used

- Black & white images (long runs of same pixel)
- Fax machines, PCX, BMP files

⚠️ **Common Mistake:** RLE makes file BIGGER for noisy/varied images. Only good for repetitive data.

🎯 Part C: ARITHMETIC CODING ⭐⭐⭐

🎯 **Essence:** *Arithmetic Coding = Map an entire message to a SINGLE NUMBER between 0 and 1.*

🧠 **Memory Trick: "Whole sentence → one decimal number"** — Each symbol shrinks the interval; final interval gives a unique number.

Algorithm

1. Start with interval [0, 1)
2. Divide it into sub-intervals based on symbol probabilities (in fixed order)
3. Pick the sub-interval matching the FIRST symbol → new interval
4. Sub-divide that interval the same way
5. Pick sub-interval for SECOND symbol → new interval
6. Repeat for every symbol

7. Final interval → pick ANY number inside it as the code

📖 Solved Numerical: Arithmetic Coding (PDF slides 48-51)

Problem: Encode message "babc" using {'a': 0.2, 'b': 0.5, 'c': 0.3}

Initial interval [0, 1) divided by probabilities:

```
a: [0.0,  0.2)    width 0.2
b: [0.2,  0.7)    width 0.5
c: [0.7,  1.0)    width 0.3
```

Step 1: First symbol "b" → use [0.2, 0.7)

Subdivide [0.2, 0.7) (width = 0.5):

```
a: [0.2, 0.2 + 0.5×0.2) = [0.20, 0.30)
b: [0.3, 0.3 + 0.5×0.5) = [0.30, 0.55)
c: [0.55, 0.55 + 0.5×0.3) = [0.55, 0.70)
```

Step 2: Second symbol "a" → use [0.20, 0.30)

Subdivide [0.20, 0.30) (width = 0.1):

```
a: [0.20, 0.22)
b: [0.22, 0.27)
c: [0.27, 0.30)
```

Step 3: Third symbol "b" → use [0.22, 0.27)

Subdivide [0.22, 0.27) (width = 0.05):

```
a: [0.22, 0.23)
b: [0.23, 0.255)
c: [0.255, 0.27)
```

Step 4: Fourth symbol "c" → use [0.255, 0.270)

✅ Final Answer

Final interval: [0.255, 0.270)

Pick any number inside, e.g., **0.26** → encodes the entire message "babc"!

Summary Table (PDF slide 51)

Interval	Subinterval			Symbol
[0, 1]	[0.0-0.2]	[0.2-0.7]	[0.7-1.0]	—
[0.2-0.7]	[0.2-0.3]	[0.3-0.55]	[0.55-0.7]	b
[0.2-0.3]	[0.2-0.22]	[0.22-0.27]	[0.27-0.3]	a
[0.22-0.27]	[0.22-0.23]	[0.23-0.255]	[0.255-0.27]	b
[0.255-0.27]	final			c

TEMPLATE FOR EACH STEP:
new_low = current_low + (current_width × cum_prob_BEFORE symbol)
new_high = current_low + (current_width × cum_prob_INCLUDING symbol)

📡 Part D: DPCM (Differential Pulse Code Modulation)

🎯 **Essence:** *DPCM = Store the DIFFERENCE between consecutive samples, not the actual values. Differences are small → fewer bits needed.*

🧠 **Memory Trick: "Don't store the height — store the step!"** If pixel values are 100, 102, 101, 103... store differences: 100, +2, −1, +2.

Why It Works

- Adjacent pixels are usually similar (interpixel redundancy!)
- Small differences need fewer bits than full pixel values
- Used in JPEG for DC components of each 8×8 block

Quick Example

```
Original pixels:   200, 201, 199, 200, 202
Differences:       200, +1, −2, +1, +2
Store: 200 (full) + small numbers → much less data
```

Where Used

- JPEG (for DC coefficients across blocks)
- Lossless JPEG mode
- Audio compression

📝 Session 4 Recap

Technique	Key Idea
HUFFMAN	Frequent → short code; build tree, find L_{avg}, %saved
RLE	AAAAA → A5
ARITHMETIC	Whole message → one number in [0, 1)
DPCM	Store differences, not values

> 🧠 **Memory Trick:** Master Mnemonic: **"HRAD"** lossless = Huffman, RLE, Arithmetic, DPCM

🎯 Likely Exam Questions

1. Construct Huffman code + find L_{avg}, CR, %saved (numerical). ⭐⭐⭐
2. Encode message using Arithmetic coding (numerical). ⭐⭐⭐
3. Compare Huffman vs Arithmetic coding. ⭐⭐
4. Explain RLE with example. ⭐⭐
5. What is DPCM? How does it remove interpixel redundancy? ⭐⭐

⚡ Session 5: Lossy Compression

IGS Quantization & Vector Quantization

🎨 Part A: IGS Quantization (Improved Gray Scale)

> 🎯 **Essence:** *IGS = Smarter way to reduce bits per pixel by adding pseudo-random noise BEFORE quantizing — hides the ugly contour edges.*

The Problem It Solves

When you reduce 256 gray levels (8 bits) → 16 levels (4 bits), you get 2:1 compression but the image looks **grainy** with visible **contour bands**. Eye notices these edges. Ugly.

> 🧠 **Memory Trick: "IGS = Inject Grainy Stuff (noise) to hide the contours"** — Add a little controlled noise → eye can't see sharp boundaries between gray levels.

IGS Algorithm

1. Take the LOWER 4 BITS (LSBs) of PREVIOUS pixel
2. ADD them to CURRENT pixel's value
3. Take the UPPER 4 BITS (MSBs) of the SUM → that's the IGS code
4. **EXCEPTION:** If current pixel's MSBs are all 1s (1111), just add 0000 (don't add anything)

📖 Solved Numerical: IGS Quantization (PDF slide 43)

Working through the example

Pixel	Gray Level	Sum	IGS Code
i − 1	N/A	0000 0000	N/A
i	0110 1100	0110 1100	0110
i + 1	1000 1011	1001 0111	1001
i + 2	1000 0111	1000 1110	1000
i + 3	1110 0100	1110 0100	1111

Walk through pixel i+1

```
Current pixel: 1000 1011
Previous pixel's LSBs (from pixel i): 1100
Sum = 1000 1011 + 0000 1100 = 1001 0111
IGS code = upper 4 bits of sum = 1001 ✅
```

Why exception for "all 1s"?

If MSBs are 1111, the value is already at max (white). Adding more would overflow. So we skip the addition.

Result

- 8-bit pixels → 4-bit IGS codes = **2:1 compression**
- No visible contour bands (noise breaks them up)
- Lossy — original can't be perfectly recovered

⚠️ **Common Mistake:** Forgetting the "all 1s" exception. Using current pixel's LSBs instead of **previous** pixel's LSBs.

🧰 Part B: Vector Quantization (VQ)

🎯 **Essence:** *VQ = Replace each block of pixels with the INDEX of a similar block stored in a "codebook." Send just the index, not the whole block.*

> 🧠 **Memory Trick: "VQ = look up a CATALOG"** — instead of describing your sofa, you just say "Item #122 from IKEA catalog."

The Big Idea

1. Break image into small blocks (e.g., 4×4)
2. Build a CODEBOOK = a dictionary of ~256 typical blocks
3. For each image block, find the CLOSEST match in codebook
4. Store only the INDEX (e.g., "block #122") instead of all 16 pixel values

📖 Solved Numerical: Vector Quantization (PDF slides 56-59)

Example 10.3.4: VQ without codebook stored

Given: 8-bit image, 256×256 pixels. Codebook of 256 entries. Block size = 4×4.

Calculation:

```
Each block originally = 16 pixels × 1 byte = 16 bytes
Each block now = 1 byte (just an index 0-255)
Compression Ratio = 16:1 ✅
```

Example 10.3.5: VQ WITH codebook stored

Step 1: Number of blocks in image

```
(256 / 4) × (256 / 4) = 64 × 64 = 4,096 blocks
Bytes for indices = 4,096 × 1 = 4,096 bytes
```

Step 2: Codebook size

```
256 entries × 16 bytes/entry = 4,096 bytes
```

Step 3: Total compressed size

```
4,096 + 4,096 = 8,192 bytes
```

Step 4: Original size

```
256 × 256 = 65,536 bytes
```

Step 5: Compression ratio

```
CR = 65,536 / 8,192 = 8:1
```

Including codebook **halved** the compression (16:1 → 8:1).

🔑 TWO VQ SCENARIOS
VQ without storing codebook → CR = 16:1
VQ WITH storing codebook → CR = 8:1

When is VQ Useful?

- Generic codebook for similar images (e.g., faces, X-rays)

- Lots of repetitive patterns
- Trade-off: smaller index → more compression but more error

📝 Session 5 Recap

Technique	Key Idea
IGS Quantization	Add prev pixel's LSBs before quantizing → hides contours → 2:1, lossy
Vector Quantization	Replace block with codebook index → 16:1 (no codebook) or 8:1 (with codebook)

🎯 Likely Exam Questions

1. Explain IGS quantization with example. ⭐⭐⭐
2. Explain Vector Quantization. How is compression achieved? ⭐⭐⭐
3. Numerical: VQ compression ratio with/without codebook. ⭐⭐⭐
4. Difference between scalar and vector quantization. ⭐⭐

⚡ Session 6: JPEG Compression

THE MOST IMPORTANT TOPIC — almost guaranteed in exam

🎯 Why JPEG?

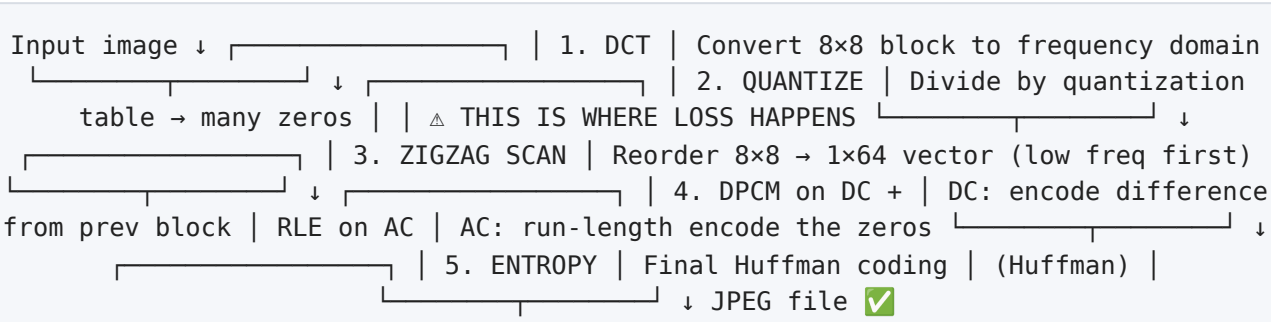
🎯 **Essence:** *JPEG = LOSSY image compression that exploits the fact that humans don't notice high-frequency details.*

Two Key Observations (PDF slide 61)

1. Image content changes **SLOWLY** across pixels → most useful info is in LOW frequencies; high frequencies = small details + noise
2. Humans are **LESS sensitive** to high-frequency loss → can SAFELY THROW AWAY high frequencies → image still looks fine!

🧠 **Memory Trick: "JPEG = throw away what eyes don't see"**

🏗️ The JPEG Pipeline (memorize this!)



🧠 **Memory Trick:** Master Mnemonic: **"DQZ-DR-H"** = **D**CT → **Q**uantize → **Z**igzag → **D**PCM/**R**LE → **H**uffman

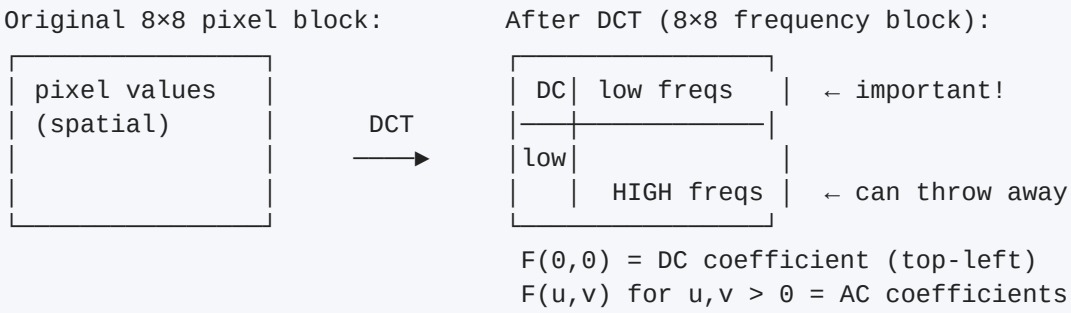
🌊 Step 1: DCT (Discrete Cosine Transform)

🎯 **Essence:** *DCT converts an 8×8 block from SPATIAL domain (pixel values) to FREQUENCY domain (how fast pixels change).*

🧠 **Memory Trick: "DCT = sorts pixel info by SPEED of change"** — Slow changes → low frequencies (top-left). Fast changes → high frequencies (bottom-right).

DSIP Module 5 Revision Guide — Page 30 of 43

What You Get After DCT



Key Terms

- **DC coefficient** = F(0,0) = average brightness of block (lowest frequency, top-left)
- **AC coefficients** = F(0,1) ... F(7,7) = all other 63 values

1D-DCT Formula

$$F(\omega) = (a(u)/2) \Sigma_{n=0}^{N-1} f(n) \cdot \cos[(2n+1)\omega\pi / 16]$$

where $a(0) = 1/\sqrt{2}$, $a(p) = 1$ for $p \neq 0$

2D-DCT (PDF slide 65)

Apply 1D-DCT on rows first, then columns. Same final result. Two series of 1-D transforms = a 2-D transform.

📉 Step 2: Quantization (where LOSS happens!)

> 🎯 **Essence:** *Divide each DCT coefficient by a number from a quantization table → small values become 0.*

$$F'(u,v) = \text{round}(F(u,v) / Q(u,v))$$

> 🧠 **Memory Trick: "Quantization = ROUND off small numbers to ZERO"** — High-freq values are small → divided by large Q values → become 0.

Why It Works

- Low frequencies (top-left): divided by small numbers → values preserved
- High frequencies (bottom-right): divided by big numbers → mostly become 0
- Result: 8×8 block has lots of zeros, especially at bottom-right!

Standard JPEG Luminance Quantization Table

```
16  11  10  16  24  40  51  61
12  12  14  19  26  58  60  55
14  13  16  24  40  57  69  56
14  17  22  29  51  87  80  62
18  22  37  56  68 109 103  77
24  35  55  64  81 104 113  92
49  64  78  87 103 121 120 101
72  92  95  98 112 100 103  99
↑ Smaller (preserve)        ↓ Bigger (kill)
```

⚠️ **Common Mistake:** This is the ONLY lossy step in JPEG baseline. Once you round off, you can't get those values back. Quality vs file-size trade-off lives HERE.

〰️ Step 3: Zigzag Scan

🎯 **Essence:** *Convert the 8×8 block to a 1×64 vector by reading in zigzag order — putting all important values first and zeros at the end.*

🧠 **Memory Trick: "ZIGZAG = sweep low-to-high frequencies"** — Reads top-left first, ends at bottom-right. Zeros pile up at the end → great for run-length encoding!

```
8×8 quantized block:          Zigzag path:
┌──────────────────┐
│ 1   2   6   7    │          1 → 2  6 → 7
│ 3   5   8  13    │          ↓ ↗  ↗ ↙
│ 4   9  12  ...   │          3   5   8
│ 10 11  ...       │          ↗ ↙   ↗
│ ...              │          4   9 ...
└──────────────────┘          ↓ ↗
                              10 → 11 ...

Result: 1×64 vector like:
[40, 12, 0, 0, 0, 0, 0, 0, 10, -7, -4, 0, 0, ..., 0]
       └─ all the zeros bunch up at the END ─┘
```

📡 Step 4a: DPCM on DC Components

Each block's DC value is replaced by the **difference** from the previous block's DC.

```
Block DC values:    45,  54,  48,  32,  ...
DPCM difference:    45,  +9,  -6, -16,  ...
                     └──┘  └──────────┘
                  send full   small differences (fewer bits!)
                  for first
```

📊 Step 4b: RLE on AC Components

The 63 AC coefficients have many zeros. Encode runs of zeros:

```
AC vector: 12, 10, 1, -7, 0, 0, -4, 0, 0, 0, 0, ..., 0

RLE pairs (skip, value):
(0, 12)   ← 0 zeros, then 12
(0, 10)   ← 0 zeros, then 10
(0, 1)    ← 0 zeros, then 1
(0, -7)   ← 0 zeros, then -7
(2, -4)   ← 2 zeros, then -4
(0, 0)    ← END-OF-BLOCK marker
```

🌳 Step 5: Entropy Coding (Huffman)

DC encoding (slide 72)

DC difference encoded as **(SIZE, Value)**:

```
Example: DC = 40, previous DC = 48
Difference = -8

Need to represent -8:
- SIZE = 4 bits
- Code for SIZE 4 (from Huffman table) = "101"
- Value bits for -8 = "0111"

Final code: 101 0111
```

AC encoding (slide 74)

AC pairs: **(RunLength/SIZE, Value)**:

```
Example: AC = 12 with 0 zeros before:
- RunLength = 0, SIZE = 4
- Code for "0/4" from Huffman table = "1011"
- Value 12 = "1100"
- Final = 1011 1100

End marker (0,0) → coded as "1010"
```

🗜️ JPEG Modes ⭐⭐⭐

Mode	What It Does
Sequential (Baseline)	Standard mode. Encode entire image one 8×8 block at a time, left-to-right, top-to-bottom. Supports 8-bit images.
Lossless	No DCT, no quantization! Uses predictive coding + Huffman. Truly lossless but lower compression.
Progressive	Send a low-quality version first, then improve in stages. Good for slow internet — see image early. Two types: **Spectral Selection** (DC first, then ACs gradually); **Successive Approximation** (MSBs first, then more bits).
Hierarchical	Send multiple resolutions of same image. Receiver picks the resolution it needs. Like 720p / 1080p / 4K versions.

> 🧠 **Memory Trick:** JPEG modes: **"SLPH"** = Sequential, Lossless, Progressive, Hierarchical (or "Some Like Pizza Hot" 🍕)

🌟 Lossless JPEG Mode Details (PDF slides 76-77)

For truly lossless JPEG, no DCT! Uses **predictive coding**:

```
For each pixel X, predict it using neighbors:

   ┌─────┬─────┐
   │  C  │  B  │      7 Predictors:
   ├─────┼─────┤      P1 = A
   │  A  │  X  │      P2 = B
   └─────┴─────┘      P3 = C
                      P4 = A + B − C
                      P5 = A + (B−C)/2
                      P6 = B + (A−C)/2
                      P7 = (A+B)/2

Notes:
- Pixel (0,0) uses itself
- First row pixels use P1
- First column pixels use P2
- The best (of 7) is chosen for each pixel
```

Encode the difference between predicted and actual → small numbers → Huffman → tiny lossless file.

🎯 Likely Exam Questions

1. **Explain JPEG compression with block diagram (almost guaranteed!)** ⭐⭐⭐
2. Explain DCT and its role in JPEG. ⭐⭐⭐

3. Explain quantization in JPEG. How does it cause loss? ⭐⭐⭐
4. Explain zigzag scan with diagram. ⭐⭐
5. Explain JPEG modes (Sequential/Progressive/Hierarchical/Lossless). ⭐⭐⭐
6. Why is JPEG lossy? Where does the loss occur? ⭐⭐
7. Compare JPEG vs lossless techniques. ⭐⭐

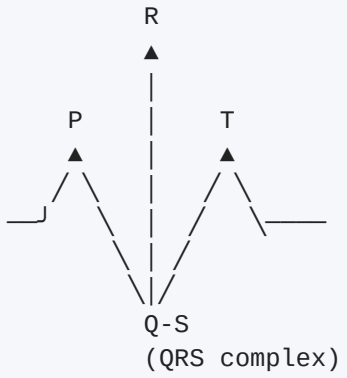
⚡ Session 7: Case Studies

ECG · MRI Tumor · Ultrasound — Real-world DSIP applications

🫀 Case Study 1: ECG Signal Analysis & Arrhythmia Detection

> 🎯 **Essence:** *Use DSP techniques to analyze heart electrical signals and detect abnormal rhythms (arrhythmia).*

The Classic ECG Waveform




```
Components:
- P wave: atrial contraction
- QRS complex: ventricular contraction (most prominent)
- T wave: ventricular relaxation
```

The DSP Pipeline for ECG

Step	What Happens
1. Preprocessing (Filtering)	Remove baseline drift, power line noise (50/60 Hz). Bandpass filter 0.5–40 Hz.
2. R-Peak Detection	Detect QRS complex (highest peak). Pan-Tompkins algorithm is famous.
3. Feature Extraction	Heart rate, RR interval, PR interval, QRS width, etc.
4. Classification	Compare features to normal/abnormal patterns. ML/Neural Networks often used.

Common Arrhythmias Detected

- **Bradycardia:** heart rate too slow (< 60 bpm)
- **Tachycardia:** heart rate too fast (> 100 bpm)
- **Atrial Fibrillation:** irregular rhythm

- **Premature beats:** extra heartbeats

DSIP Techniques Used

- Filtering (FIR/IIR) → remove noise
- Wavelet transform → multi-resolution analysis
- FFT → frequency components
- Threshold detection → find R-peaks

🧠 Case Study 2: Tumor Segmentation in MRI Images

> 🎯 **Essence:** *Use image processing to identify and isolate tumor regions in MRI scans.*

What's MRI?

Magnetic Resonance Imaging — uses magnetic fields to produce detailed images of body tissues. Tumors usually appear as **abnormally bright or dark regions**.

The Pipeline

Step	What Happens
1. Preprocessing	Noise removal (Gaussian filter), contrast enhancement (histogram equalization)
2. Skull Stripping	Remove non-brain regions using morphological operations
3. Segmentation	Separate tumor from healthy tissue. Methods: Thresholding, K-means, Region growing, Watershed
4. Post-Processing	Morphological ops: **Opening** (remove noise), **Closing** (fill holes), **Boundary extraction** (outline tumor)
5. Feature Analysis	Tumor size, shape, texture → benign vs malignant

> 💡 **Tip:** This case study perfectly demonstrates how Module 5 morphology operations (Opening, Closing, Boundary Extraction) are essential in real medical applications!

Where Module 5 Concepts Are Used

- **Morphological operations** (opening, closing) → clean segmented tumor
- **Boundary extraction** → outline tumor edges
- **Image enhancement** → improve tumor visibility
- **Edge detection** → find tumor boundaries

🩺 Case Study 3: Ultrasound Image Enhancement

> 🎯 **Essence:** *Improve the quality of ultrasound images, which are typically grainy and have low contrast (speckle noise).*

Why Ultrasound Is Tricky

- **Speckle noise:** granular pattern from sound wave interference
- **Low contrast:** tissues look similar
- **Shadows:** from bones/gas blocking sound waves

The Pipeline

Step	What Happens
1. Noise Removal	Speckle noise → Median filter, Wiener filter, Wavelet denoising
2. Contrast Enhancement	Histogram equalization, Adaptive histogram (CLAHE)
3. Edge Enhancement	Sharpen edges of organs/structures using Sobel, Laplacian filters
4. Segmentation (optional)	Isolate organ of interest. Often uses morphological operations

Common Applications

- Pregnancy scans (fetal monitoring)
- Cardiac imaging (heart structures)
- Abdominal scans (liver, kidney)

🎯 The Common Thread

Module Used	Where Applied
Filtering	Noise removal in all 3 cases
Histogram processing	Contrast in MRI, ultrasound
Edge detection	Tumor boundaries, organ edges
Morphology (Module 5)	Tumor segmentation cleanup
Compression (Module 5)	Storage of medical images (DICOM)
Wavelet transform	ECG analysis, denoising
Frequency analysis	ECG, tissue characterization

DSIP Module 5 Revision Guide — Page 38 of 43

📝 How to Answer Case Study Exam Questions

For 5-mark questions:

1. Define the application (1-2 lines)
2. Draw a block diagram of the pipeline
3. Briefly explain each block (1 line each)
4. Mention DSIP techniques used (filtering, morphology, etc.)
5. State final outcome (diagnosis/enhancement)

For 10-mark questions, also add:

- Challenges (e.g., speckle noise)
- Specific algorithms (e.g., Pan-Tompkins for QRS)
- Real-world impact

🎯 Likely Exam Questions

1. Explain ECG signal analysis with block diagram. ⭐⭐⭐
2. Explain tumor segmentation in MRI using image processing. ⭐⭐⭐
3. How is ultrasound image enhancement done? Explain steps. ⭐⭐
4. Discuss applications of morphological operations in medical imaging. ⭐⭐
5. What are the challenges in ECG signal processing? ⭐

🏆 Master Summary & Exam Strategy

Your last-minute revision arsenal

✅ Module 5 Complete Checklist

Topics Covered:

- ✅ Dilation & Erosion (binary morphology basics)
- ✅ Opening & Closing (compound operations)
- ✅ Hit-or-Miss Transform (template matching)
- ✅ Boundary Extraction
- ✅ Compression: redundancies, ratio, fidelity
- ✅ Lossless: Huffman, RLE, Arithmetic, DPCM
- ✅ Lossy: IGS, Vector Quantization
- ✅ JPEG full pipeline + 4 modes
- ✅ Case studies: ECG, MRI, Ultrasound

🏆 TOP 10 Most Likely Exam Questions

#	Question	Priority
1	Numerical: Dilation/Erosion of given image with SE	⭐⭐⭐
2	Numerical: Huffman coding (build tree, find L_{avg}, %save)	⭐⭐⭐
3	Diagram: JPEG compression pipeline with explanation	⭐⭐⭐
4	Numerical: Compression ratio calculation	⭐⭐⭐
5	Theory: 3 types of redundancies (CIP)	⭐⭐⭐
6	Numerical: Hit-or-Miss transform	⭐⭐⭐
7	Theory: Opening vs Closing (with diagram)	⭐⭐⭐
8	Numerical: Arithmetic coding interval narrowing	⭐⭐⭐
9	Theory: JPEG modes (Sequential/Progressive/Lossless)	⭐⭐
10	Theory: Case study (ECG / MRI tumor / Ultrasound)	⭐⭐

🧠 The Ultimate Memory Dump

MORPHOLOGY:

- **Dilation:** ANY overlap → ON (grow) — "Donation"
- **Erosion:** ALL match → ON (shrink) — "Eraser"
- **Opening:** Erode then Dilate (cleans outside)
- **Closing:** Dilate then Erode (fills inside)
- **Hit-Miss:** 2 erosions, then AND
- **Boundary:** $A - (A \ominus B)$

COMPRESSION BASICS:

- $CR = U_{size} / C_{size}$
- $R_D = 1 - 1/CR$
- 3 redundancies: **CIP** (Coding, Interpixel, Psychovisual)
- Fidelity: Objective (RMS, SNR) | Subjective (humans)

LOSSLESS — "HRAD":

- **Huffman:** Frequent → short (build tree)
- **RLE:** AAAA → A4
- **Arithmetic:** message → number in [0, 1)
- **DPCM:** store differences

LOSSY:

- **IGS:** add prev LSBs, take MSBs (2:1)
- **VQ:** codebook lookup (16:1 or 8:1)
- **JPEG:** DQZ-DR-H pipeline

JPEG MODES — "SLPH":

- Sequential, Lossless, Progressive, Hierarchical

💡 Top Tips for Your Exam

1. **For numericals:** Show ALL steps clearly — partial marks save you
2. **For diagrams:** Label every block, draw arrows neatly
3. **For theory:** Use bullet points, include 1 example
4. **For Huffman:** Don't worry about 0/1 convention — just be consistent
5. **For JPEG:** ALWAYS draw the full pipeline diagram even if not asked
6. **For redundancy types:** Memorize "CIP" with 1-line example each
7. **For morphology:** Practice on actual grids (paper) before exam
8. **Time management:** Numericals first if you're confident, theory second

🎯 Quick Formula Reference Card

Concept	Formula
Dilation	$A \oplus B = \{ x \mid (\hat{B})_x \cap A \neq \varnothing \}$
Erosion	$A \ominus B = \{ x \mid (B)_x \subseteq A \}$
Opening	$A \circ B = (A \ominus B) \oplus B$
Closing	$A \bullet B = (A \oplus B) \ominus B$
Hit-or-Miss	$HM(X, B) = (X \ominus B) \cap (X^c \ominus (W - B))$
Boundary	$\beta(A) = A - (A \ominus B)$
Compression Ratio	$C_R = U_{size} / C_{size}$
Relative Redundancy	$R_D = 1 - (1/C_R)$
Average Code Length	$L_{avg} = \Sigma P_i \times L_i$
Entropy	$H = - \Sigma P_i \times \log_2(P_i)$
RMS Error	$e_{rms} = [(1/MN) \Sigma \Sigma (\hat{f} - f)^2]^{1/2}$
Quantization (JPEG)	$F'(u,v) = round(F(u,v) / Q(u,v))$

🎉 You're Ready! 🎉

Module 5 is now fully in your head.

Stay calm, read questions carefully, and show your steps.

Best of luck for your exam! 🚀

DSIP Module 5 Revision Guide — Page 43 of 43